

Aura + FIAP

SKILLS & GO

Agentic Engineering

Bem vindos

Prof. Dr. Ives Santos

Bacharel - Química

Mestre - Química Analítica

Doutor - Química Computacional

Especialista II
SENAI/CIMATEC

Professor
Fiap



Prof. Ives Santos



Quem são vocês?

Sobre o curso

Alura + FIA/P

Fundamentos e práticas para a construção de fluxos agênticos com grandes modelos de linguagem (LLMs): projetar, prototipar, avaliar e evoluir fluxos de sistemas de agentes de IA de maneira responsável e fundamentada.

Aula 01

Fundamentos
Componentes
LLM
Contexto e RAG

Aula 02

Onde criar um agente
Frameworks
Graphos
Prompt intection

Aula 03

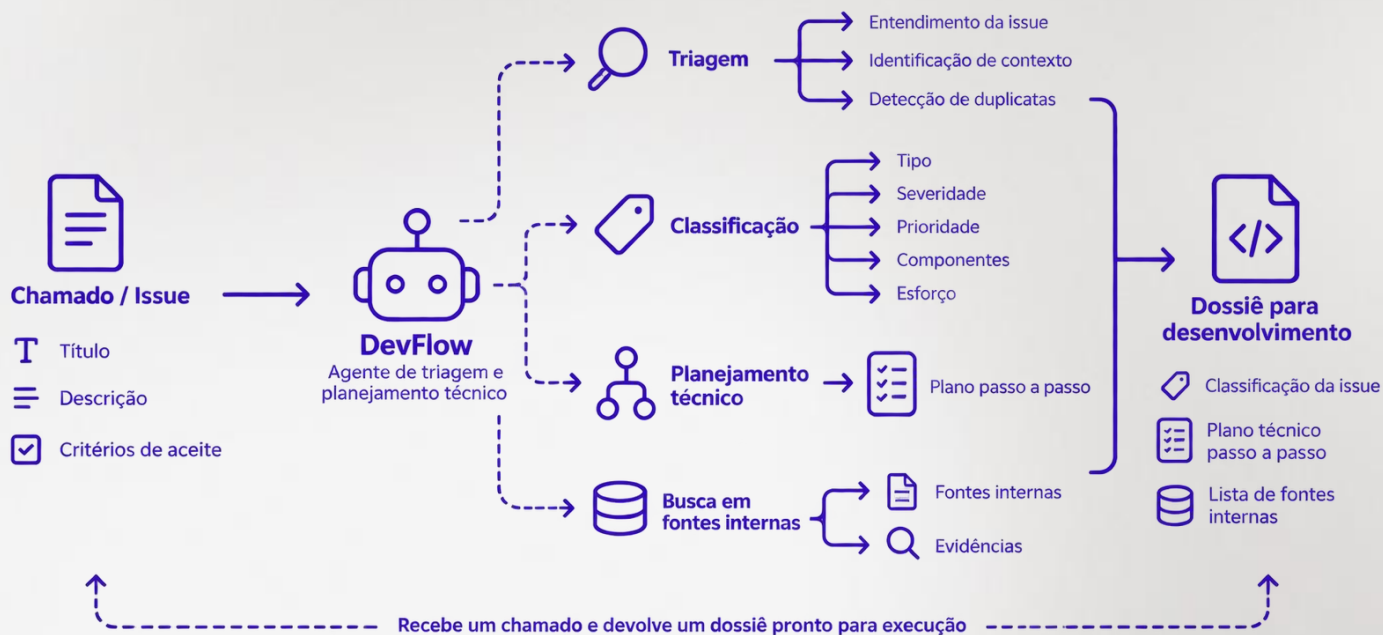
Guardrails
Human-in-the-loop
Multiagentes

Aula 04

LLMOps
Monitoramento
Deploy

Projeto Final DevFlow

O DevFlow é um agente de **triagem e planejamento técnico de issues de software**. Ele recebe um chamado escrito por outra pessoa e devolve um dossiê pronto para uma pessoa desenvolvedora executar.



Material da Aula

Criar um diretório para o curso
No terminal, entrar no diretório
`$ git clone <repositório>`

API KEY: OpenAI

Conta Azure

Aula 01

Fundamentos

Prof. Ives Santos

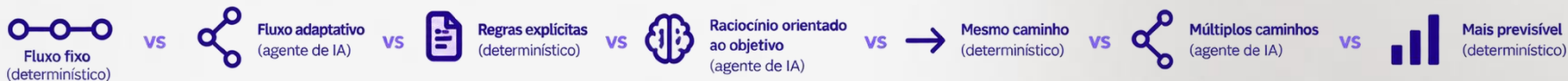
WorkFlow Determinístico - Automação

No workflow determinístico o caminho já está definido; no agente autônomos o modelo decide como executar a tarefa dentro dos limites e objetivos

Workflow determinístico



Agente autônomo



Exemplo de Sistemas Agênticos

Interação com usuário

Agentes que interagem com interfaces visuais humanas, interpretando cliques, imagens e

Automação de Fluxo de Trabalho

Plataformas que conectam sistemas, APIs e gatilhos para automatizar fluxos de trabalho de ponta a ponta. Exemplo: Automação de processos entre CRM, e-mail e banco de dados via IA – sem precisar escrever código.

Desenvolvimento

Auxiliam desenvolvedores escrevendo, corrigindo ou explicando código, muitas vezes usando múltiplos LLMs. Exemplo: Agentes que criam pipelines CI/CD automaticamente a partir de requisitos descritivos.

O que é um Agente de IA?

Agente é um sistema de software que usa inteligência artificial para perceber o ambiente, tomar decisões de forma autônoma e executar ações usando ferramentas para alcançar uma meta específica.

Autonomia

É a capacidade de o agente decidir e executar uma sequência de passos sem que uma pessoa precise comandar cada etapa.

Ação

É a capacidade de interagir com o ambiente, e não apenas gerar texto.

Objetivo

É o resultado que o agente precisa alcançar.

Raciocínio

É a capacidade de decidir o que fazer para atingir o objetivo.

Exemplo de Sistemas Agênticos



Rede Mater Dei de Saúde

12 agentes de IA no ciclo de receita hospitalar



Cox Automotive

17 soluções agênticas em produção



Uber for Business

Agente para leads, follow-up e conversão



Rakuten

Codex em desenvolvimento, CI/CD e resposta a incidentes

Harness: Conjunto de código, configuração e lógica de execução que dá ao modelo capacidades que ele não tem sozinho, como manter estado, executar ferramentas, acessar conhecimento atualizado e operar em ambientes reais.

Agent = Model + Harness

O modelo propõe

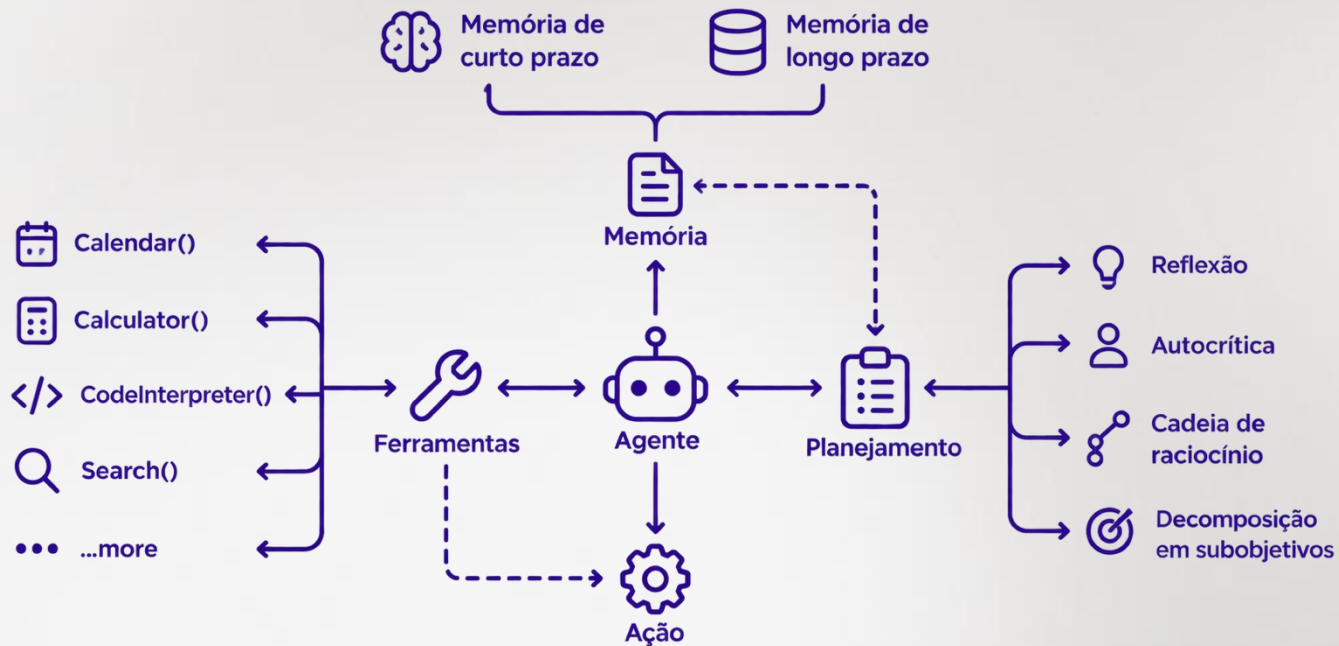
- interpreta intenção
- prioriza possibilidades
- seleciona ação permitida
- redige uma saída

O Harness garante

schemas e validação
limites, retries e timeout
autorização e sandbox
logs, traces e avaliação

Componentes

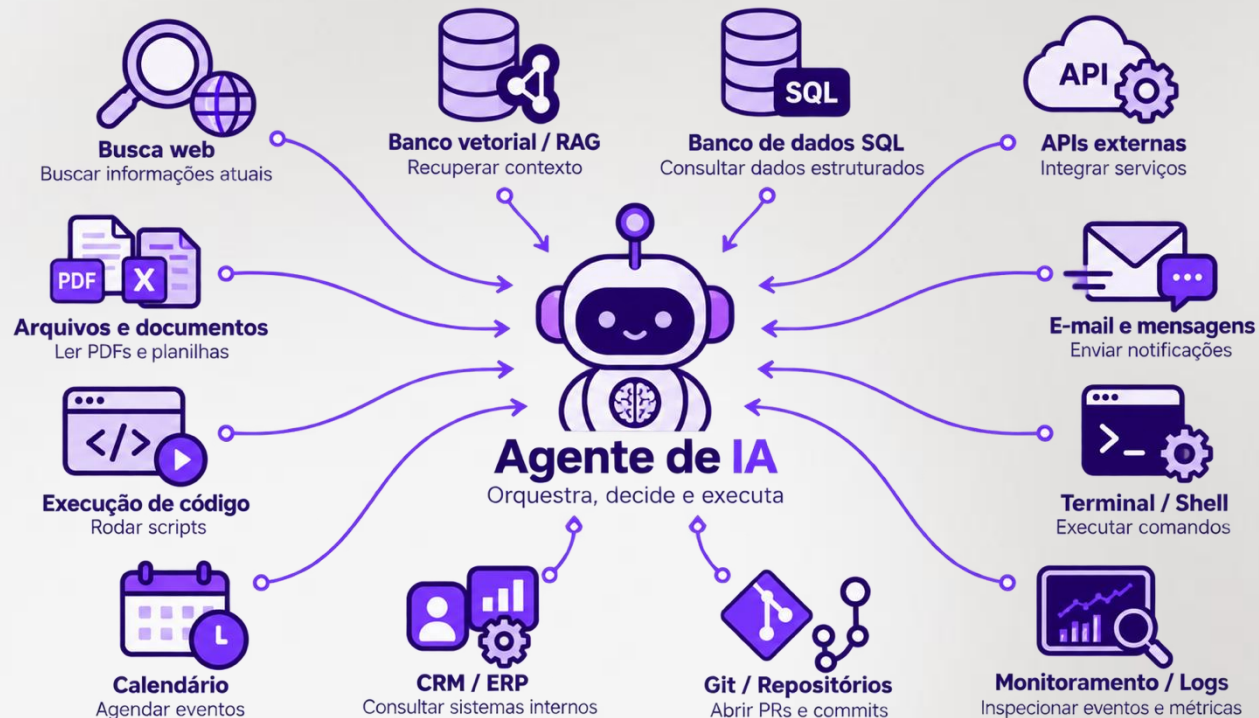
Os principais componentes de um agente de IA podem ser organizados assim:



Tools



Interfaces, APIs, códigos ou softwares externos que um agente pode acionar deliberadamente para interagir com o mundo digital e realizar tarefas que vão além da sua capacidade nativa de processamento de texto.



Escrever uma função de código (geralmente em Python) e fornecer uma descrição em texto claro explicando o que essa função faz.

Ferramenta com Crewai

```
from crewai.tools import tool
```



```
@tool("Calculadora de Desconto Progressivo")
```

```
def calcular_desconto(valor_total: float) -> str:
```

```
    """
```

```
    Calcula o desconto de uma compra com base no valor total.
```

```
    Use esta ferramenta sempre que o cliente perguntar o preço final ou o
```

```
    """
```

```
    if valor_total >= 500:
```

```
        preco_final = valor_total * 0.85
```

```
        return f"Desconto de 15% aplicado! Preço final: R$ {preco_final:.2f}"
```

```
    elif valor_total >= 200:
```

```
        preco_final = valor_total * 0.90
```

```
        return f"Desconto de 10% aplicado! Preço final: R$ {preco_final:.2f}"
```

```
    else:
```

```
        return f"Sem desconto para valores abaixo de R$ 200. Preço final: R$ {valor_total:.2f}"
```

Ferramenta com LangChain

```
from langchain.tools import tool
```



```
@tool
```

```
def search_database(query: str, limit: int = 10) -> str:
```

```
    """Search the customer database for records matching the query.
```

```
    """
```

```
    Args:
```

```
        query: Search terms to look for
```

```
        limit: Maximum number of results to return
```

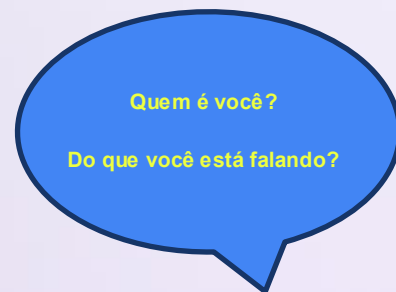
```
    """
```

```
    return f"Found {limit} results for '{query}'"
```

DEMONSTRAÇÃO

Lab_01_A_ferramentas: Construindo ferramentas com o LangChain @tools
Google Colab

Memória



Memória

Alura + FIAP

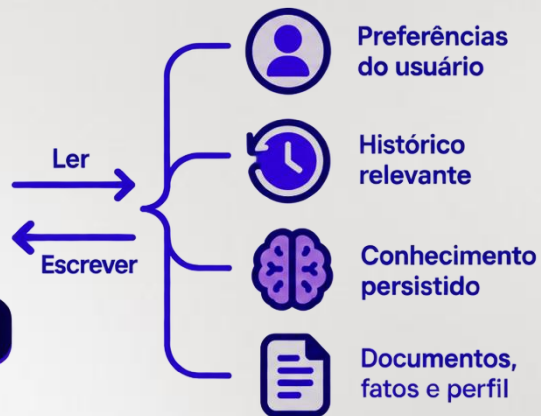
Arquitetura cognitiva ou o sistema persistente que permite ao agente acumular conhecimento, reter o contexto de interações e adaptar seu comportamento ao longo do tempo

Memória de curto prazo



Temporária • rápida • limitada

Memória de longo prazo



Persistente • reutilizável • escalável

Ferramentas



redis



LangChain



Flask

Bancos de dados Relacionais



Banco de Vetores



Armazenamento



amazon
S3



Prof. Ives Santos

Reasoning



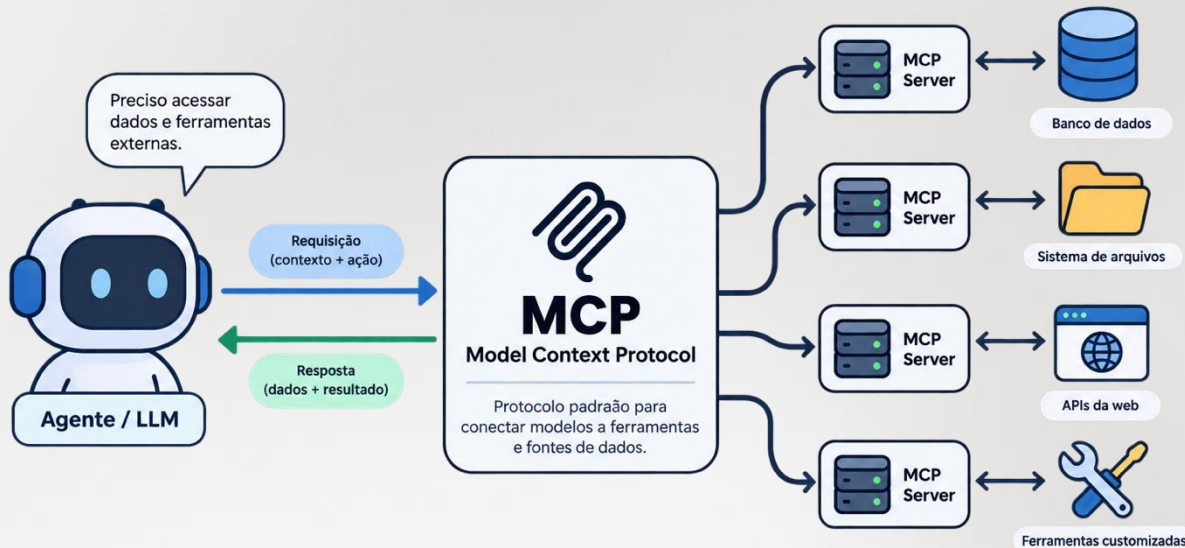
Reasoning

O "cérebro" central do sistema que processa informações, avalia cenários e toma decisões de forma autônoma para atingir um objetivo específico



MCP – Model Context Protocol

Como o raciocínio pode acionar ferramentas ?



O que é?

O MCP (Model Context Protocol) é um protocolo aberto que define um padrão para que modelos de IA se conectem a ferramentas, dados e sistemas externos.



Como funciona?

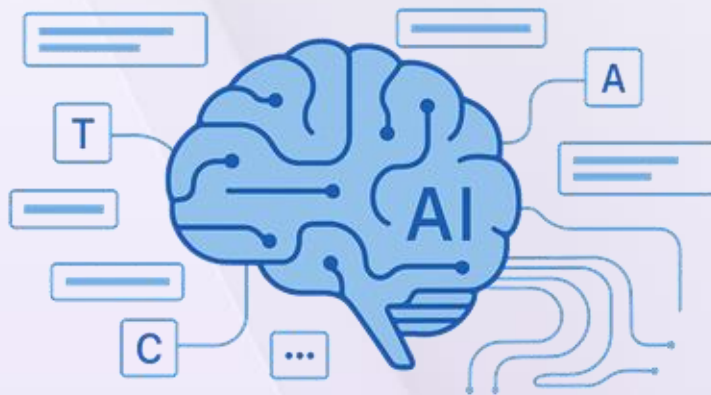
- 1 O agente faz uma requisição via MCP
- 2 O MCP direciona para o servidor certo
- 3 O servidor acessa a ferramenta ou dados
- 4 O resultado volta para o agente



Benefícios

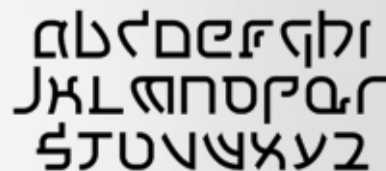
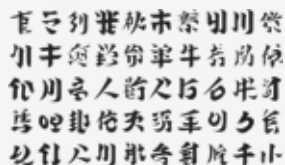
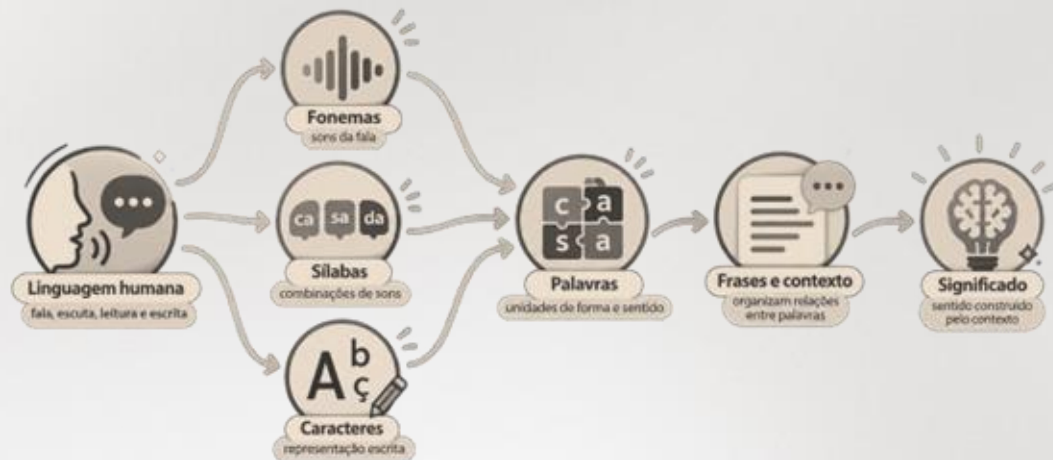
- ✓ Padronização de integrações
- ✓ Mais segurança e controle
- ✓ Facilita criação de tools
- ✓ Ecossistema crescente

LLMs



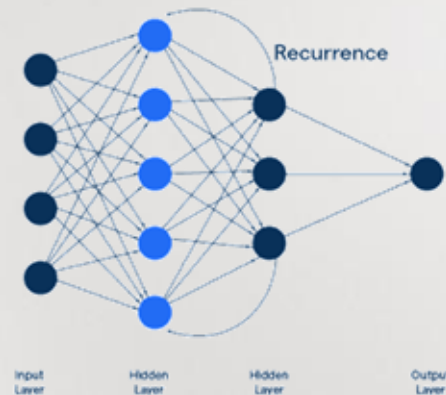
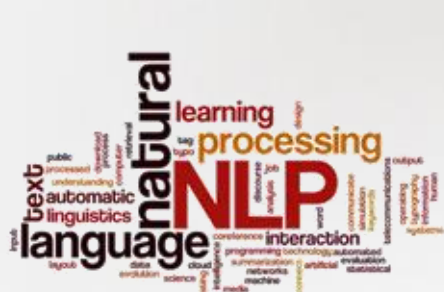
Linguagem

A linguagem não é apenas um amontoado de palavras, mas um sistema complexo que as máquinas tentam simular matematicamente.



NLP Processamento de Linguagem Natural

“Campo da Inteligência Artificial (IA) que combina linguística, ciência da computação e machine learning para permitir que máquinas compreendam, interpretem e gerem linguagem humana (texto ou fala).”



Arquitetura Transformer

Arquitetura de rede neural criada para processar sequências de dados, como texto, usando principalmente o mecanismo de **Attention**, em vez de depender de processamento sequencial como RNNs ou LSTMs.

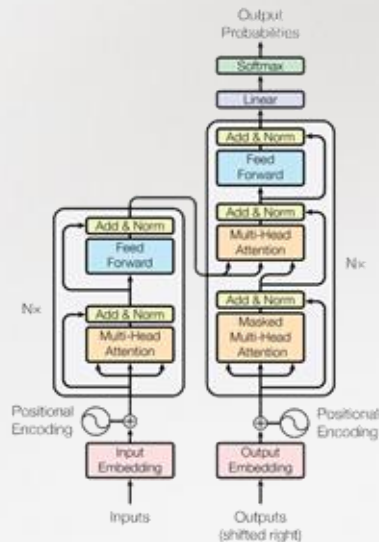


Figure 1: The Transformer - model architecture.

Attention Is All You Need

Ashish Vaswani*
Google Brain
avaswani@google.com

Noam Shazeer*
Google Brain
noam@google.com

Niki Parmar*
Google Research
nikip@google.com

Jakob Uszkoreit*
Google Research
usz@google.com

Llion Jones*
Google Research
llion@google.com

Aidan N. Gomez*[†]
University of Toronto
aidan@cs.toronto.edu

Lukasz Kaiser*
Google Brain
lukaszkaizer@google.com

Illia Polosukhin*[‡]
illia.polosukhin@gmail.com

Abstract

The dominant sequence transduction models are based on complex recurrent or convolutional neural networks that include an encoder and a decoder. The best performing models also connect the encoder and decoder through an attention mechanism. We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely. Experiments on two machine translation tasks show these models to be superior in quality while being more parallelizable and requiring significantly less time to train. Our model achieves 28.4 BLEU on the WMT 2014 English-to-German translation task, improving over the existing best results, including ensembles, by over 2 BLEU. On the WMT 2014 English-to-French translation task, our model establishes a new single-model state-of-the-art BLEU score of 41.8 after training for 3.5 days on eight GPUs, a small fraction of the training costs of the best models from the literature. We show that the Transformer generalizes well to other tasks by applying it successfully to English constituency parsing both with large and limited training data.

Processamento de Grandes Modelos

Alura + FIAP

Por que esses modelos surgiram agora?



Corpus

Base de dados massiva



Arquitetura Transformer

Modelo mais eficiente para linguagem

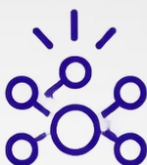


Hardware

Poder computacional



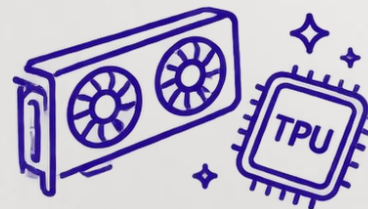
Tokens



Atenção



Saída



Bilhões de documentos



Web, livros, código e artigos



Grandes volumes de texto



Captura contexto



Processa em paralelo



Escala melhor



GPUs e TPUs



Treinamento em larga escala

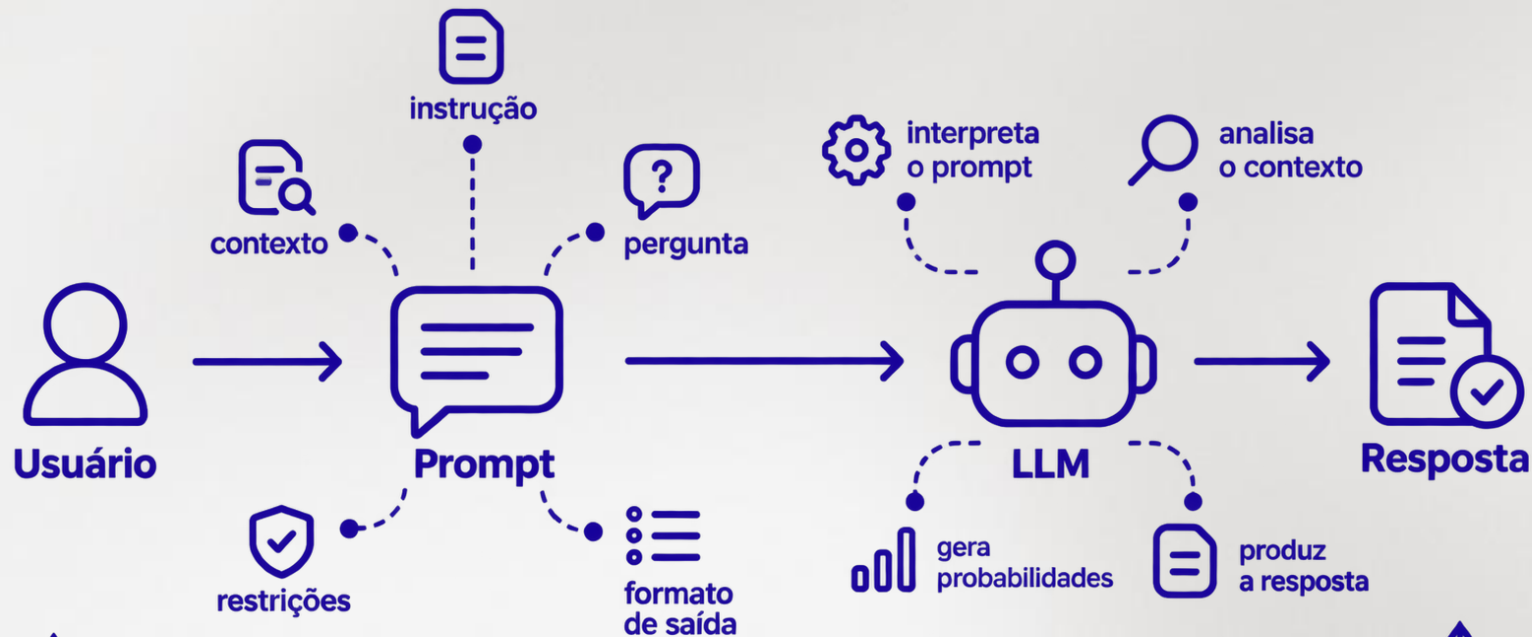


Mais memória e velocidade

Prof. Ives Santos

Interação com os Grandes Modelos

Por que esses modelos surgiram agora?



A forma de conversar com um modelo LLM é por meio de prompts bem estruturados.

Exemplo: "Resuma este texto em 3 tópicos."

IA as a Service

Alura + FIA/P

Os modelos podem ser consumidos via API pagando pelo input e output de tokens



DEMONSTRAÇÃO

Lab_01_B_provedores: chamando GPT, Gemini e Opus (dados, metadados, janela de contexto, custo)

IA on-premisse

Você pode processar LLMs localmente executando os pesos do modelo na sua própria máquina ou servidor, em vez de chamar uma API externa

Modelos Opoen

Llama

Qwen

Mistral

Gemma

DeepSeek

Baixar os pesos

Hugging Face

Ollama

Runtime

Ollama — simples para desenvolvimento local

LM Studio — interface gráfica

Transformers — maior controle via código

Dúvidas?

Quizz



Voltamos em

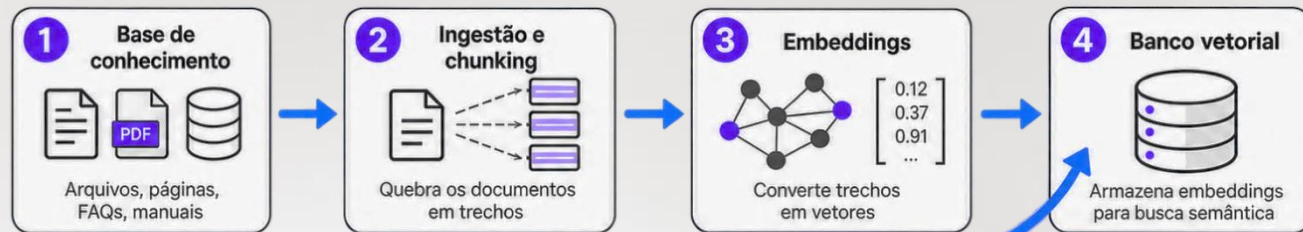
RAG

Pipeline

O RAG acontece através da execução de algumas etapas

PIPELINE DE INGESTÃO

Obter os dados, tratar e deixa-los disponíveis para serem consultados.



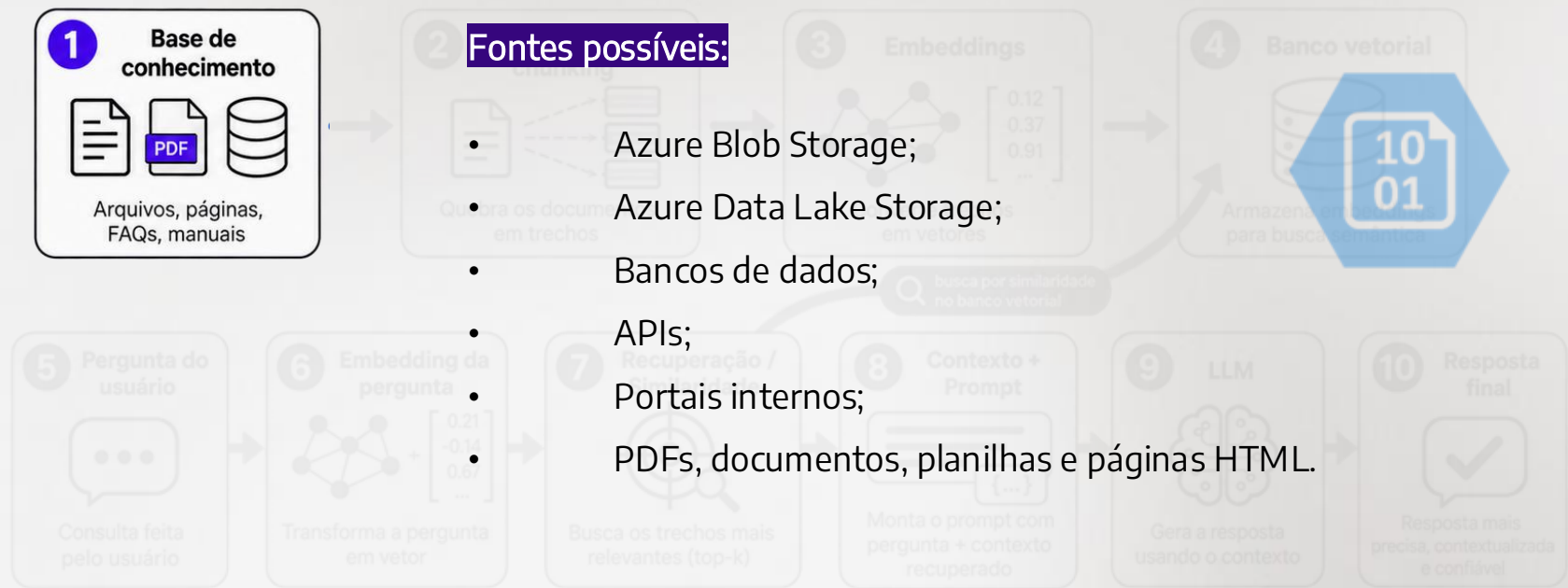
PIPELINE RECUPERAÇÃO/CONSULTA

Buscar os dados mais relevantes para a consulta:



Pipeline

Base de dados



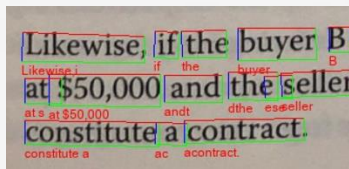
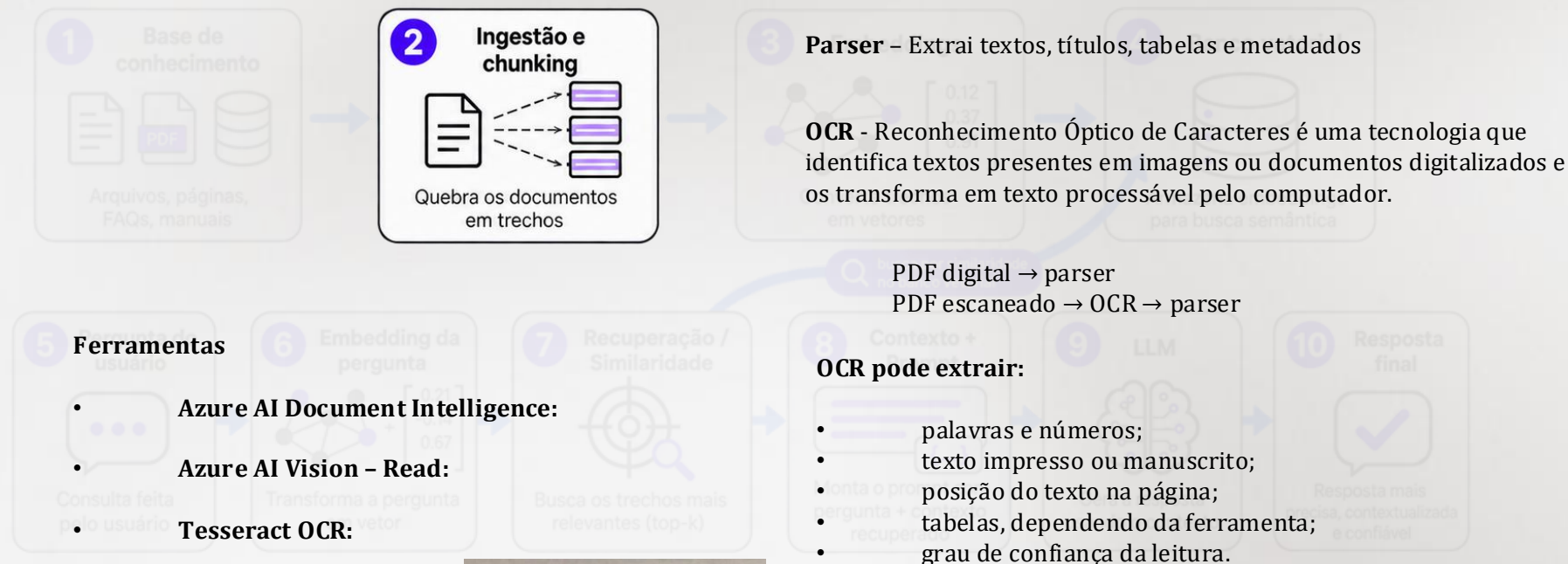
Pipeline

Chunks



Pipeline

Chunks



Estratégias

Tamanho fixo

Exemplo: 500 tokens por chunk – overlap 50 tokens

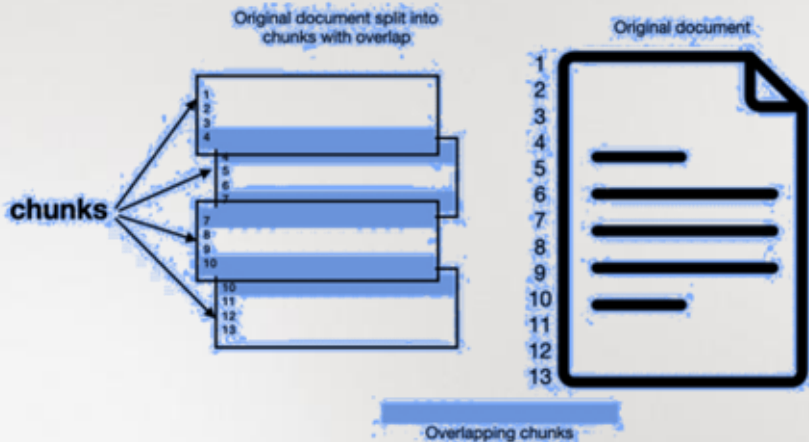
Chunk Semântico

O chunking semântico é uma técnica de ponta projetada para segmentar o texto em grupos significativos e conceitualmente distintos

Divisão Recursivo/ Estrutural

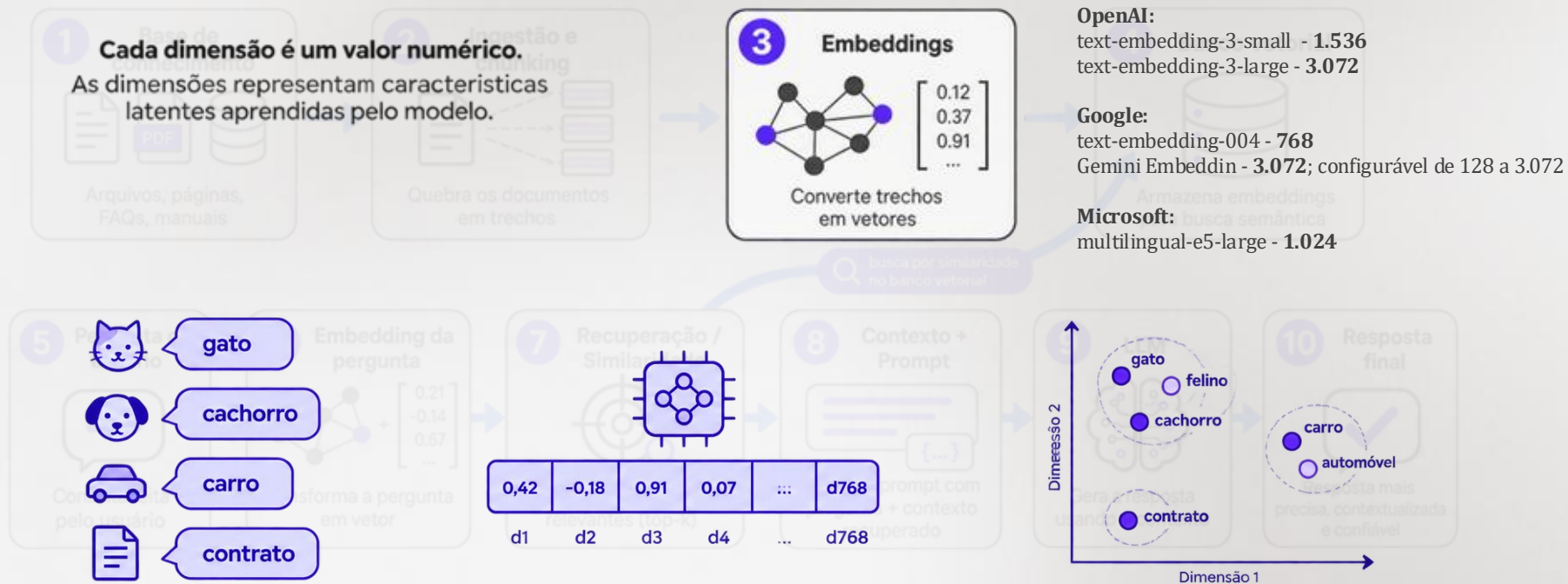
Utiliza a organização do documento:

- capítulos;
- títulos e subtítulos;
- parágrafos;
- artigos e cláusulas;
- perguntas e respostas.



Pipeline

São representações numéricas de textos em forma de vetores. Eles capturam significado semântico.



OpenAI:

text-embedding-3-small - 1.536

text-embedding-3-large - 3.072

Google:

text-embedding-004 - 768

Gemini Embeddin - 3.072; configurável de 128 a 3.072

Microsoft:

multilingual-e5-large - 1.024

DEMONSTRAÇÃO

Lab_01_C_token_embeddings: tokenização e embeddings

Pipeline

O RAG acontece através da execução de algumas etapas

Exemplos Azure

Azure IA Search

Azure Cosmos DB

Azure Database for PostgreSQL com pgvector

Exemplos fora da Azure

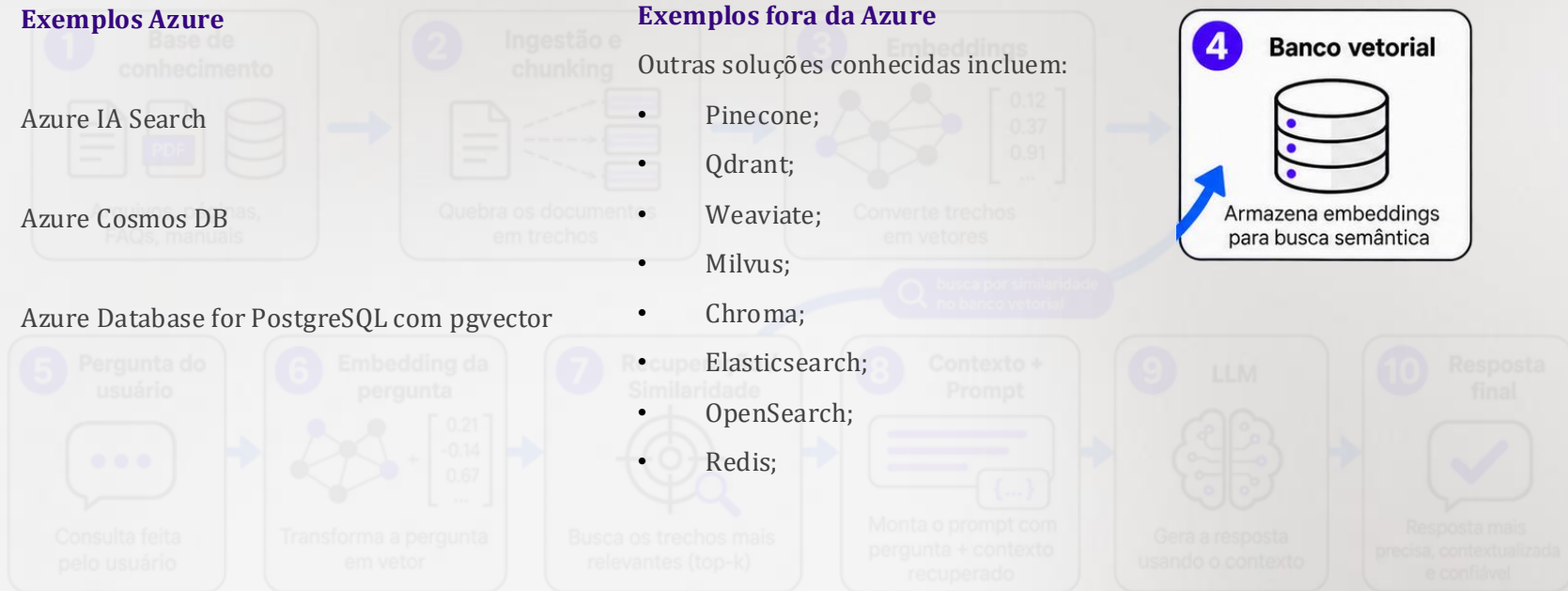
Outras soluções conhecidas incluem:

- Pinecone;
- Qdrant;
- Weaviate;
- Milvus;
- Chroma;
- Elasticsearch;
- OpenSearch;
- Redis;

4 Banco vetorial



Armazena embeddings para busca semântica

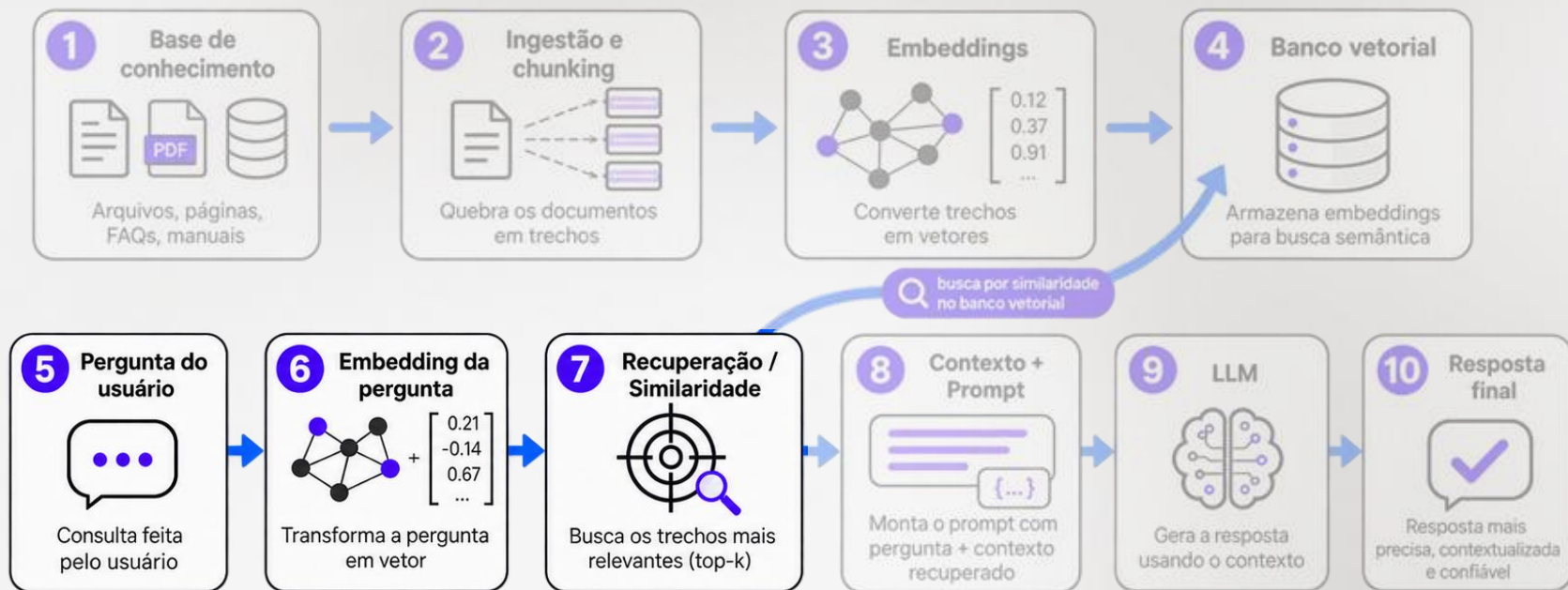


DEMONSTRAÇÃO

Lab_01_D_azaisearch: indexação no Azure AI Search

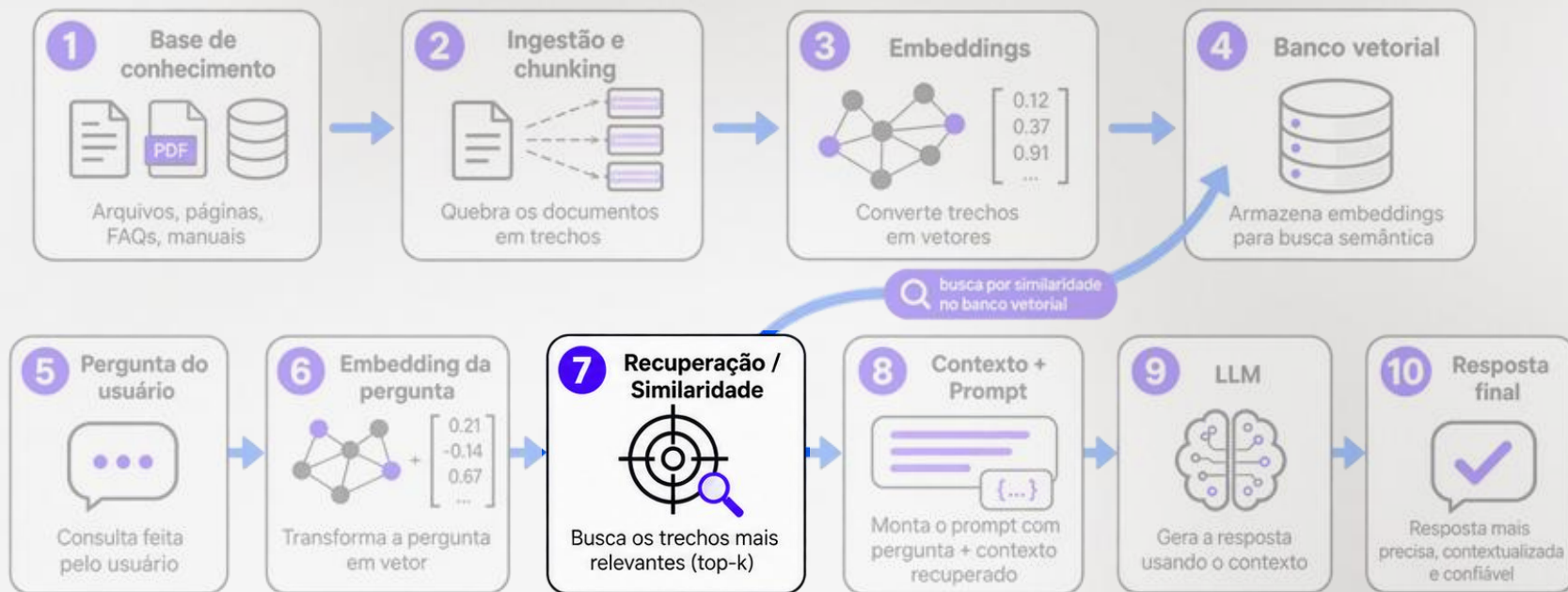
Pipeline

O RAG acontece através da execução de algumas etapas



Pipeline

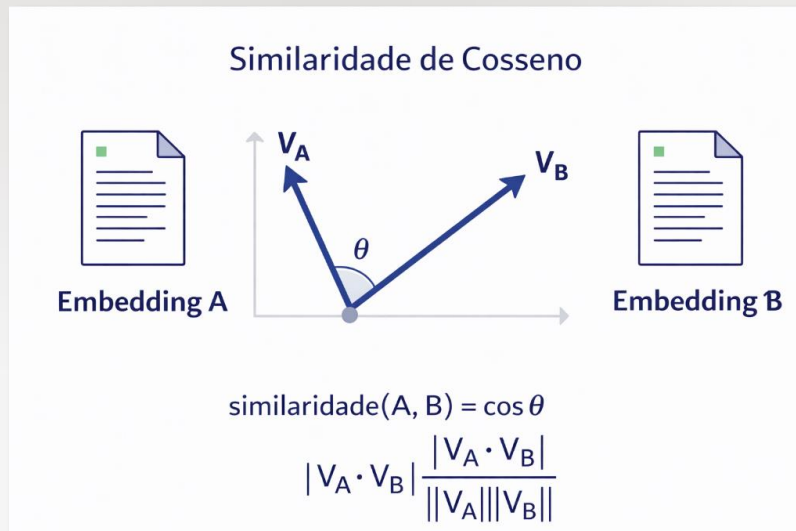
O RAG acontece através da execução de algumas etapas



Similaridade

Similaridade entre vetores pode ser calculada por diferentes métricas. Em aplicações com embeddings, as mais comuns são Similaridade de Cosseno, Produto Escalar e Distância Euclidiana.

- **Similaridade de Cosseno** — mede o ângulo entre os vetores, ignorando em grande parte sua magnitude. Muito comum em embeddings e busca semântica.
- Distância Euclidiana.
- Distância Manhattan (L1) .
- Produto Escalar (Dot Product)
- Distância de Minkowski
- Similaridade de Jaccard
- Distância de Hamming



DEMONSTRAÇÃO

Lab_01_E_similaridade: cálculo de similaridade de cosseno, comparação entre vetores.

PRÁTICA

Scores e Calibração

Alura + FIAP

Scores e métricas servem para medir se o mecanismo de recuperação do RAG está trazendo o contexto certo, na quantidade certa e com desempenho aceitável.

Precision@k	Recall@k	Taxa de perguntas sem contexto:	Cobertura de fontes:	Latência de recuperação
Entre os k documentos recuperados, quantos realmente são relevantes.	Entre todos os documentos relevantes existentes, quantos apareceram nos k resultados recuperados.	Percentual de consultas para as quais o sistema não encontrou evidências suficientes.	Verifica se o sistema está conseguindo recuperar informações das diferentes bases/fontes disponíveis.	Tempo necessário para localizar e retornar o contexto.

A calibração consiste em **ajustar parâmetros** como k, threshold de similaridade, filtros e reranking para encontrar um equilíbrio entre qualidade, cobertura, custo e velocidade.

Imagine uma pergunta:

“Quais documentos tratam sobre LGPD?”

* Antes do teste, alguém define que os documentos relevantes são:

PDF-01 ; PDF-02 ; DOC-01 ; XLS-01

Esse conjunto vira a referência para avaliar o RAG.

Se o sistema executar a busca com k=5 e retornar:

PDF-01 
PDF-02 
PDF-05 
DOC-01 
XLS-01. 

$\text{Precision@5} = 4 / 5 = 80\%$

$\text{Recall@5} = 4 / 4 = 100\%$

Ex2. Agora imagine que existiam 6 documentos relevantes na base, mas encontramos apenas 3:

PDF-01 
PDF-07 
PDF-05 
DOC-01 
XLS-08. 

$\text{Recall@5} = 2 / 4 = 50\%$



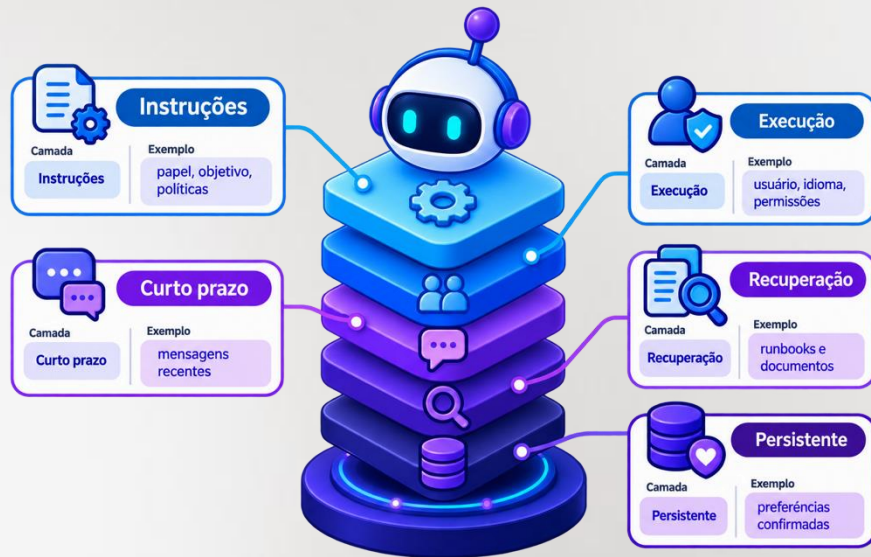
Tamanho do Contexto

Informações relevantes versus ruído: Uma boa estratégia maximiza evidência útil e minimiza ruído, duplicação e conteúdo não autorizado.

Mais contexto pode piorar

Contexto excessivo pode causar:

- Distração;
- Conflito de instruções;
- Evidência perdida no meio;
- Mais tokens;
- Mais latência;
- Maior exposição de dados.



Prompts e instruções orientadas a tarefas



Prompt orientado à tarefa

Todo prompt de produção deve explicitar:

1. Objetivo
2. Entradas
3. Restrições
5. Formato

"system", "Você é analista de SRE da SkillGo.

OBJETIVO

Classificar o incidente e recomendar o próximo passo operacional mais seguro.

REGRAS

1. Use somente fatos presentes no relato do incidente.
2. Não invente métricas, causas, comandos ou resultados.
3. Separe fatos observados de hipóteses.
4. Se faltarem dados críticos, faça no máximo duas perguntas objetivas.
5. Nunca recomende uma ação destrutiva sem revisão humana.

FORMATO

Responda com: Resumo, Fatos, Hipóteses, Próximo passo e Incertezas.""
("human", "INCIDENTE\n{incidente}"),

Dúvidas?

Quizz



Para a próxima aula

Como podemos criar e fazer todos esses componentes interagirem?

